



PERTEMUAN 11

Exception Handling dalam Java

Pelajari cara membuat program Java yang tangguh — mampu mendeteksi, menangani, dan pulih dari kesalahan secara elegan tanpa menghentikan jalannya aplikasi.





Pengenalan Exception Handling

Dalam dunia pemrograman nyata, kesalahan selalu bisa terjadi — mulai dari input yang salah, file yang tidak ditemukan, hingga koneksi jaringan yang terputus. **Exception** (pengecualian) adalah sinyal bahwa sesuatu yang tidak terduga terjadi saat program berjalan.

🚨 Menangkap Masalah

Exception menunjukkan adanya masalah selama eksekusi program, seperti pembagian dengan nol atau file tidak ditemukan.

🛡️ Tetap Berjalan

Dengan exception handling, program bisa terus berjalan seolah tidak ada masalah, atau berhenti secara anggun tanpa crash mendadak.

🛡️ Program Lebih Kuat

Fitur ini membantu membangun aplikasi yang *robust* (kuat) dan *fault-tolerant* (toleran terhadap kesalahan).

Kapan Menggunakan Exception Handling?

Exception handling dirancang khusus untuk menangani **kesalahan sinkron** — yaitu kesalahan yang terjadi langsung saat sebuah pernyataan dieksekusi. Ini berbeda dengan peristiwa asinkron seperti klik mouse atau pesan jaringan yang terjadi secara terpisah dari alur program utama.

Indeks Array Keluar Batas

Mengakses elemen di luar ukuran array yang telah ditentukan.

Pembagian dengan Nol

Operasi aritmatika yang menghasilkan nilai tidak terdefinisi.

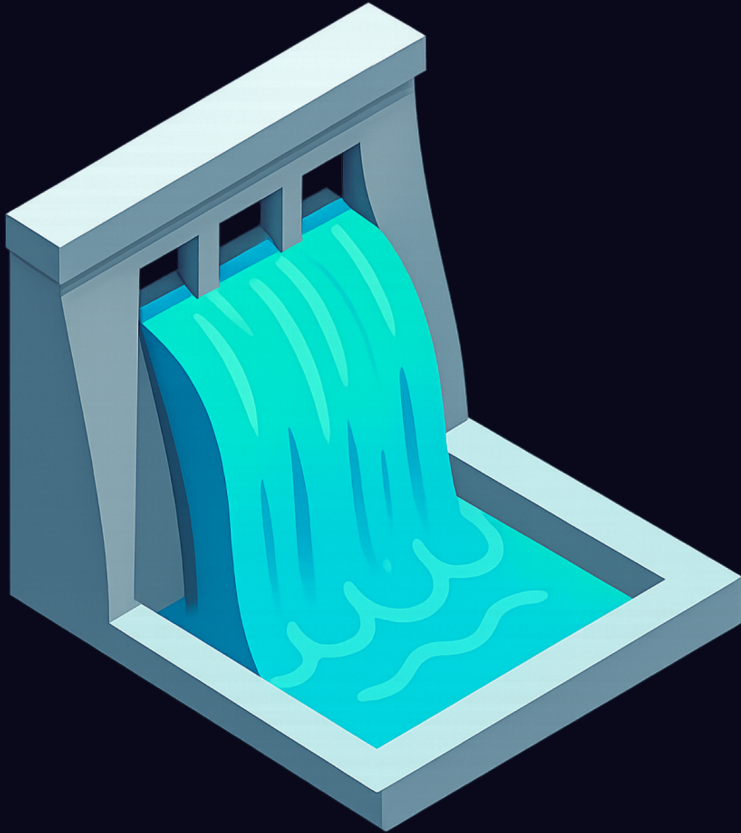
Parameter Tidak Valid

Memanggil metode dengan argumen yang tidak sesuai tipe atau nilainya.

Arithmetic Overflow

Nilai hasil perhitungan melebihi batas representasi tipe data yang digunakan.

 Exception handling **tidak** dirancang untuk menangani peristiwa asinkron seperti klik mouse, penekanan tombol, atau kedatangan pesan jaringan.



Finally Block — Blok yang Selalu Dieksekusi

Blok **finally** adalah bagian kode yang *selalu* dijalankan setelah blok try-catch selesai, baik ada exception maupun tidak. Tujuan utamanya adalah memastikan sumber daya dikembalikan ke sistem dengan benar untuk menghindari **kebocoran sumber daya** (*resource leaks*).

1

🔍 Proses Berjalan

Blok try dijalankan. Program berusaha menyelesaikan tugasnya.

2

⚠️ Exception Terjadi?

Blok catch menangkap dan menangani pengecualian yang muncul.

3

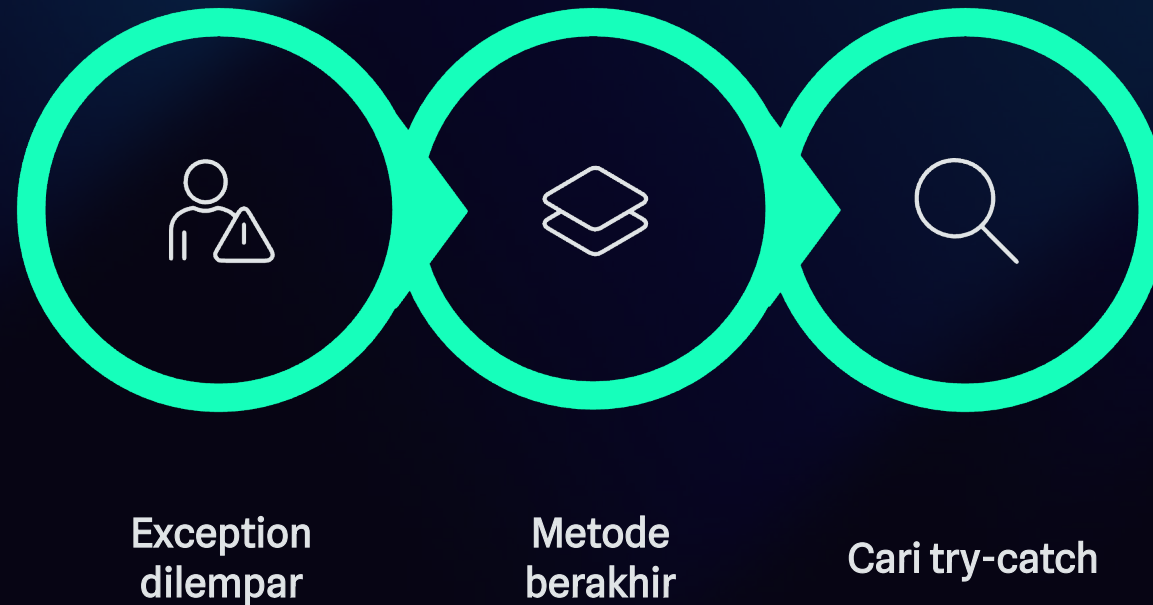
✅ Finally Selalu Jalan

Blok finally dieksekusi untuk menutup file, koneksi database, atau jaringan.

Java memang melakukan *garbage collection* otomatis untuk memori, namun sumber daya seperti **file**, **koneksi database**, dan **koneksi jaringan** harus ditutup secara manual agar tidak bocor ke program lain.

Stack Unwinding — Melepas Tumpukan Panggilan

Stack unwinding terjadi ketika sebuah exception dilempar tapi tidak ditangkap di dalam metode yang sedang berjalan. Java kemudian "melepas" tumpukan panggilan — mundur satu per satu ke metode pemanggil — sampai menemukan blok `try-catch` yang bisa menangani exception tersebut.



Bayangkan seperti tumpukan buku yang diambil satu per satu dari atas hingga menemukan buku yang dicari. Setiap metode yang "dibuka" akan membersihkan variabel lokalnya dan menyerahkan kendali ke pemanggil. Jika tidak ada blok `try` yang cocok ditemukan hingga metode `main`, program akan berhenti dengan pesan error.



Chained Exceptions — Rantai Pengecualian

Pernakah Anda mencoba memperbaiki bug tapi tidak tahu akar masalahnya karena pesan errornya tidak jelas? Itulah masalah yang diselesaikan oleh **Chained Exceptions** (pengecualian berantai).

1 Masalah Lama

Jika blok catch melempar exception baru, informasi exception asli dan jejak tumpukan (*stack trace*) akan **hilang**. Ini membuat debugging sangat sulit karena kita tidak tahu di mana masalah sebenarnya terjadi.

2 Solusi: Chained Exception

Dengan chained exception, objek exception baru dapat **membungkus** exception asli di dalamnya. Sehingga jejak tumpukan lengkap dari exception awal tetap tersedia untuk ditelusuri.

3 Manfaat Debugging

Developer bisa menelusuri rantai exception dari yang terakhir hingga ke akar penyebab pertama, membuat proses pelacakan bug jauh lebih mudah dan efisien.

Deklarasi Exception Types

Dalam pengembangan aplikasi Java, kita sering menggunakan kelas-kelas yang sudah tersedia dari **Java API**, vendor pihak ketiga, atau pustaka yang dapat diunduh secara bebas. Metode-metode dalam kelas tersebut sudah dirancang untuk melempar exception yang sesuai saat masalah terjadi.



Java API

Java menyediakan ratusan exception bawaan seperti `NullPointerException`, `ArrayIndexOutOfBoundsException`, dan `IOException` yang siap digunakan.



Library Pihak Ketiga

Library eksternal seperti Hibernate atau Spring juga memiliki exception khusus yang perlu dipahami dan ditangani oleh programmer.



Exception Kustom

Programmer dapat membuat tipe exception sendiri dengan meng-*extend* kelas `Exception` untuk kebutuhan bisnis yang lebih spesifik.

Preconditions & Postconditions

Dua konsep penting dalam desain metode yang baik adalah **precondition** dan **postcondition**. Keduanya berfungsi sebagai "kontrak" antara pemanggil metode dan implementasi metode itu sendiri.

Preconditions

Kondisi yang **harus benar sebelum** metode dipanggil. Mencakup:

- Batasan pada parameter metode
- Harapan tentang keadaan program saat ini
- Jika tidak dipenuhi, perilaku metode menjadi tidak terdefinisi

Contoh: parameter tidak boleh null, angka harus positif.

Postconditions

Kondisi yang **harus benar setelah** metode berhasil selesai.
Mencakup:

- Batasan pada nilai kembalian metode
- Efek samping yang mungkin terjadi
- Wajib didokumentasikan agar pemanggil tahu apa yang diharapkan

Contoh: hasil selalu berupa bilangan bulat positif.

Assertions — Pernyataan Validasi Program

Assertion adalah kondisi yang kita nyatakan "seharusnya benar" pada titik tertentu dalam program. Assertion sangat berguna saat proses pengembangan dan debugging untuk menangkap bug potensial sedini mungkin sebelum sampai ke pengguna.

1

Memvalidasi Logika

Assertion memastikan asumsi-asumsi logis dalam kode Anda memang benar pada saat runtime, bukan hanya saat kompilasi.

2

Menangkap Bug Lebih Awal

Dengan assertion, bug yang tersembunyi di logika program bisa terdeteksi jauh sebelum merusak data atau pengalaman pengguna.

3

Dua Jenis Assertion

Precondition assertion: memeriksa keadaan program saat metode dipanggil. **Postcondition assertion:** memeriksa keadaan setelah metode selesai berjalan.

📌 Assertion diaktifkan menggunakan flag `-ea` saat menjalankan program Java. Secara default, assertion tidak aktif di lingkungan produksi.



Dasar-Dasar Karakter dan String

Setiap program Java dibangun dari rangkaian **karakter**. Karakter adalah blok bangunan terkecil dalam kode sumber Java. Beberapa karakter digabungkan membentuk **String** — salah satu tipe data yang paling sering digunakan dalam pemrograman Java.

Literal String

Ditulis sebagai urutan karakter di antara tanda kutip ganda. Contoh: `"Halo, Dunia!"`. Literal string disimpan dalam memori sebagai objek `String`.

Deklarasi Variabel String

Literal string dapat ditugaskan ke referensi `String`. Contoh: `String nama = "Budi";` — variabel `nama` kini menunjuk ke objek `String` di memori.

String adalah Objek

Berbeda dengan bahasa lain, `String` di Java adalah objek dari kelas `java.lang.String`, bukan tipe primitif biasa. Ini berarti `String` memiliki banyak metode yang bisa dipanggil.

String Constructors

Kelas **String** menyediakan berbagai konstruktor untuk menginisialisasi objek String dengan cara yang berbeda-beda. Memahami konstruktor ini penting agar Anda bisa membuat objek String sesuai kebutuhan program Anda.

String Kosong

`new String()` – Membuat String kosong tanpa karakter apapun di dalamnya.

Dari Literal

`new String("teks")` – Membuat String baru yang berisi salinan dari literal yang diberikan.

Dari Array Char

`new String(charArray)` – Mengubah array karakter menjadi sebuah objek String yang utuh.

Dari StringBuilder

`new String(sb)` – Membuat String dari isi StringBuilder yang mutable (bisa diubah).

Meskipun bisa menggunakan `new String("teks")`, cara yang lebih efisien dan umum adalah langsung menggunakan literal: `String s = "teks";` karena Java mengoptimalkan penggunaan memorinya melalui *String Pool*.

String Methods: length, charAt, dan getChars

Kelas String menyediakan berbagai metode untuk mengeksplorasi isi dari sebuah String. Tiga metode dasar yang paling sering digunakan adalah

String Compare — Membandingkan String

Membandingkan String adalah operasi yang sangat umum dalam pemrograman. Java menyediakan beberapa cara untuk melakukan perbandingan, masing-masing dengan karakteristik yang berbeda. Penting untuk memahami perbedaannya agar tidak terjadi bug yang sulit dideteksi.



`equals()`

Membandingkan isi dua String secara tepat, membedakan huruf besar dan kecil. `"Java".equals("java") → false`



`equalsIgnoreCase()`

Membandingkan isi dua String tanpa mempedulikan huruf besar/kecil. `"Java".equalsIgnoreCase("java") → true`



`compareTo()`

Membandingkan dua String secara leksikografis.
Mengembalikan 0 jika sama, negatif jika lebih kecil, positif jika lebih besar.



Operator `==`

Membandingkan **referensi** (alamat memori), bukan isi.
Hindari menggunakan `==` untuk membandingkan isi String — gunakan `equals()` sebagai gantinya.

Extracting Substrings dari Strings

Kelas String menyediakan dua versi metode

Concatenating Strings — Menggabungkan String

Penggabungan String (**concatenation**) adalah proses menyambung dua atau lebih String menjadi satu String baru. Java menyediakan dua cara populer untuk melakukan ini:

Metode `concat()`

Metode `concat(String str)` menggabungkan dua objek String dan mengembalikan objek String baru yang berisi karakter dari kedua String asli. Contoh:

```
"Halo ".concat("Dunia")  
→ Menghasilkan: "Halo Dunia"
```

Operator `+`

Cara yang lebih umum dan ringkas adalah menggunakan operator `+`. Di balik layar, Java mengubahnya menjadi operasi `StringBuilder`. Contoh:

```
"Halo " + "Dunia"  
→ Menghasilkan: "Halo Dunia"
```

i Untuk penggabungan dalam loop yang berulang banyak kali, gunakan `StringBuilder` karena lebih efisien dalam penggunaan memori dibanding operator `+` biasa.



String Method `valueOf()`

Setiap objek di Java memiliki metode `toString()` untuk menghasilkan representasi Stringnya. Namun, tipe data primitif seperti `int`, `double`, atau `boolean` **tidak memiliki** metode karena bukan objek. Di sinilah

StringBuilder: length, capacity, setLength & ensureCapacity

StringBuilder adalah versi String yang *mutable* (bisa diubah isinya). Berbeda dengan String biasa yang immutable, StringBuilder memungkinkan modifikasi isi tanpa membuat objek baru setiap kali — sangat efisien untuk manipulasi teks yang intensif.

length()

Mengembalikan jumlah karakter yang **saat ini ada** di dalam StringBuilder. Mirip dengan `length()` pada String biasa.

capacity()

Mengembalikan jumlah karakter yang **dapat ditampung** tanpa perlu mengalokasikan memori tambahan. Kapasitas awal default adalah 16 karakter.

ensureCapacity(int min)

Menjamin bahwa StringBuilder memiliki **setidaknya** kapasitas minimum yang ditentukan. Berguna sebelum menambahkan teks dalam jumlah besar.

setLength(int newLength)

Mengatur ulang panjang StringBuilder. Jika lebih kecil dari panjang saat ini, karakter di akhir akan dipotong. Jika lebih besar, karakter null akan ditambahkan.

StringBuilder append() Methods

Metode

StringBuilder: Metode Insertion

Selain menambahkan di akhir, StringBuilder juga menyediakan metode

StringBuilder: Metode Deletion

StringBuilder juga dilengkapi dengan metode untuk **menghapus karakter**. Ada dua metode yang tersedia: `delete()` untuk menghapus rentang karakter, dan `deleteCharAt()` untuk menghapus satu karakter pada posisi tertentu.

`delete(int start, int end)`

Menghapus semua karakter dari indeks `start` hingga `end - 1`. Karakter pada indeks `end` tidak ikut dihapus.

Contoh: `sb.delete(0, 4)` menghapus 4 karakter pertama.

`deleteCharAt(int index)`

Menghapus tepat satu karakter pada indeks yang ditentukan. Semua karakter setelahnya akan bergeser ke kiri satu posisi.

Contoh: `sb.deleteCharAt(3)` menghapus karakter keempat.

⚠ Indeks Tidak Valid

Sama seperti `insert()`, kedua metode akan melempar `StringIndexOutOfBoundsException` jika indeks yang diberikan tidak valid.

Kombinasi `insert()`, `delete()`, dan `deleteCharAt()` menjadikan StringBuilder sebagai alat manipulasi teks yang sangat fleksibel dan efisien untuk kebutuhan dinamis dalam aplikasi Java.

Terima Kasih!

Semoga materi **Exception Handling** dan **String/StringBuilder** di Java ini bermanfaat dan mudah dipahami. Teruslah berlatih dan jangan takut untuk mencoba – setiap error adalah kesempatan belajar!

🔑 Poin Kunci

Exception handling membuat program lebih kuat dan toleran terhadap kesalahan.

🔗 Rantai Exception

Chained exceptions memudahkan debugging dengan mempertahankan jejak akar masalah.

📄 String & StringBuilder

Gunakan String untuk teks tetap, StringBuilder untuk manipulasi teks dinamis yang efisien.

Universitas Putera Batam — Sasa Ani Arnomo, S.Kom., M.S.I. | Mata Kuliah Pemrograman Java

